Software Developer Intern – Technical Assessment

Overview

The goal of this assessment is to evaluate your ability to:

- Build an interactive application that consumes public APIs.
- Transform raw data into meaningful visualizations.
- Design responsive interfaces that react dynamically to user input.
- Write clean, modular, and maintainable code.

You will build a **small interactive data application** using **one of the following APIs**. You are free to choose **any frontend framework, library, or language** that you are comfortable with.

Expected effort: **1–2 days of focused work**

Task Options (Choose ONE)

You may choose **one of the following tasks** based on your interest.

Expected effort: **1–2 days of focused work**

Option 1 — Pokémon Team Builder

API
https://pokeapi.co/

Goal

Create a **Pokémon team analysis tool** that helps users build and evaluate a team of Pokémon.

Your application should allow users to assemble a team of Pokémon and provide insights about **team balance, type coverage, and potential weaknesses**.

Requirements

1. Team Builder

Allow the user to:

- Search or select Pokémon from the **PokéAPI**
- Add up to **6 Pokémon** to a team
- Display basic information such as:
    - Pokémon name
    - Types
    - Base stats
    - Abilities

The team composition should update dynamically as Pokémon are added or removed.

---

2. Team Analysis

Your application should analyze the team and provide insights about its composition.

Type Coverage

Evaluate how well the team performs against different Pokémon types.

Examples:

- Highlight **team weaknesses** (e.g. “Your team is weak against Electric-type attacks”)
- Identify **duplicate weaknesses**
- Show **types the team is strong against**

You may display this as:

- A table
- A chart
- A summarized report

Exact implementation is flexible.

Stat / Role Coverage

Visualize or summarize how balanced the team is based on Pokémon stats.

Relevant stats include:

- HP
- Attack
- Defense
- Special Attack
- Special Defense
- Speed

Possible insights:

- Number of **physical attackers**
- Number of **special attackers**
- Defensive Pokémon
- Speed distribution

You may create simple heuristics to classify Pokémon roles based on their stats.

---

3. Recommendations

Provide simple suggestions based on the team composition.

Examples:

- Highlight missing roles (e.g. “Your team lacks defensive Pokémon”)
- Suggest replacing a Pokémon to reduce weaknesses
- Identify overlapping weaknesses in the team

Your recommendation logic **does not need to be perfect** — simple rules are sufficient as long as the reasoning is clear.

Pokémon Concepts (Reference)

If you are unfamiliar with Pokémon mechanics, the following resources may help explain the concepts used in this task.

Type Effectiveness

Pokémon attacks interact differently with types.

Examples:

- Water attacks are strong against Fire
- Electric attacks are strong against Water
- Ground attacks have no effect on Electric

Reference:
https://pokemondb.net/type

Type Coverage

“Coverage” refers to how well a team can attack different Pokémon types.

For example, a team may struggle against Dragon types if none of its Pokémon have strong attacks against them.

Example reference tool:
https://pokemondb.net/tools/type-coverage

You are **not required to replicate this tool**, but it demonstrates the concept.

PokéAPI Documentation

The PokéAPI provides the data needed for this task.

Documentation:
https://pokeapi.co/docs/v2

Relevant endpoints include Pokémon types, stats, and abilities.

Important Note

You **do not need to implement full competitive Pokémon mechanics**.

Simple rules and reasonable assumptions are acceptable.
Focus on demonstrating **data analysis, visualization, and logical reasoning** using the API data.

Option 2 — Singapore Hawker Centre Map

Dataset
https://data.gov.sg/datasets/d_4a086da0a5553be1d89383cd90d07ecd/view

Goal

Create an **interactive geospatial visualization** that plots Singapore hawker centres on a map.

Your application should allow users to explore the distribution of hawker centres across Singapore and interact with individual locations.

Requirements

1. Map Visualization

Plot hawker centres on a **map of Singapore** using the dataset provided.

Each hawker centre should be represented by a **map marker**.

Each marker should display basic information such as:

- Hawker centre name
- Address
- Postal code

This information should appear when a user **clicks or hovers over the marker**.

---

2. Interactivity

Your map should support basic interactions such as:

- Zoom and pan
- Click or hover popups for hawker centre information
- Searching or filtering hawker centres

Examples of useful interactions include:

- Searching hawker centres by name
- Filtering locations based on region or area
- Highlighting a selected hawker centre

The interface should update dynamically as users interact with the map.

---

3. Optional Data Insights

You may implement additional visualization features such as:

- Marker clustering to group nearby hawker centres
- Density or distribution visualization
- Region-based grouping of hawker centres

These are optional but may help improve usability and exploration.

Geospatial Data Notes

The dataset includes **latitude and longitude coordinates** for each hawker centre.

These coordinates represent the location of a place on the Earth's surface and can be used to plot markers on a map.

Example coordinate:

Latitude: 1.3521
Longitude: 103.8198

Reference:
https://www.latlong.net/latitude-and-longitude.html

Mapping Libraries

You may use **any mapping or visualization library** you prefer.

A commonly used option is:

Leaflet

Leaflet is a lightweight open-source JavaScript library for building interactive maps.

Documentation:
https://leafletjs.com/

Quick start tutorial:
https://leafletjs.com/examples/quick-start/

Leaflet works well with **OpenStreetMap**, which does not require API keys or tokens.

OpenStreetMap

OpenStreetMap provides map tiles that can be used without requiring authentication for basic use.

Reference:
https://www.openstreetmap.org/

Important Notes

- You may use **any framework, language, or mapping library**.
- Focus on **clarity, usability, and interactivity** rather than visual polish.
- If the dataset contains missing or inconsistent data, document your assumptions in your README.

Option 3 — Rick & Morty Universe Explorer

API
https://rickandmortyapi.com/

Goal

Create an **interactive data dashboard** that explores relationships between Rick & Morty characters and their origin universes.

Your application should allow users to browse characters and visualize how different characters relate to their **origin locations or universes**.

Requirements

1. Character Explorer

Fetch and display characters from the **Rick and Morty API**.

For each character, display relevant information such as:

- Character name
- Species
- Status (Alive, Dead, Unknown)
- Origin location
- Current location

Provide a way for users to **search or browse characters**.

Examples:

- Search by character name
- Filter by species
- Filter by status

---

2. Origin Universe Visualization

Create a visualization that shows how characters are distributed across different origin locations or universes.

Possible approaches include:

- Characters grouped by origin
- Character counts per origin
- Relationship between origin and current location

Your visualization may use charts, graphs, or diagrams.

Possible visualization types:

- Bar charts
- Network graphs
- Tree diagrams
- Sankey diagrams
- Any other visualization you find appropriate

The visualization should help users **understand patterns or relationships in the data**.

---

3. Interactive Filtering

Allow users to interact with the data and update the visualization dynamically.

Examples:

- Filter characters by species
- Filter characters by status
- Filter by origin location

The visualization should update **as filters change**.

API Notes

The Rick and Morty API provides character data including:

- Character metadata
- Origin location
- Current location
- Episode appearances

API documentation:
https://rickandmortyapi.com/documentation

Pagination

The API returns character data across multiple pages.

Your application should handle pagination if you want to analyze larger portions of the dataset.

Documentation:
https://rickandmortyapi.com/documentation/#pagination

Important Notes

- You may use **any programming language, framework, or visualization library**.
- Focus on **clarity, usability, and meaningful visualization** rather than visual polish.
- If the API data contains missing or inconsistent values, document your assumptions in your README.

Submission Guidelines

Deliverables:

- GitHub repository:
  - All source code
  - Dependencies or environment configuration
  - `README.md`

README must include:

Overview

- Your approach and architecture

Setup Instructions

- How to install dependencies
- How to run the app

Testing Instructions

- How to run tests

Assumptions / Challenges

- Design decisions or limitations

Evaluation Criteria

Criteria	Weight	Description
API Integration & Data Handling	25%	Correct API usage, data transformation, and handling of loading/error states
Visualization & Interactivity	25%	Quality of visualization and responsiveness to user input
UI/UX Design & Usability	30%	Clarity, usability, and interaction quality
Code Quality & Structure	15%	Clean architecture, readability, and maintainability
Documentation	5%	Clear setup instructions and explanation of design choices

---

Timeline

Deadline: **13 March 2026**

Estimated effort: **1–2 days**

Notes

- Focus on **clarity, correctness, and interactivity** over visual polish.
- Your solution does not need to be perfect — we care about **how you approach the problem**.
- Document assumptions if the API data is inconsistent.